

Actividad Individual proyecto final

Luis David Villa López

Ingeniería en analítica de datos

Universidad de Manizales

Programación I

Natalia Marcela Castellanos Gómez

17 de mayo de 2026

1. Introducción

El presente proyecto tiene como propósito aplicar los fundamentos de la Programación Orientada a Objetos a través de la simulación de un entorno real. El problema que resuelve este sistema es la gestión administrativa de un consultorio médico en crecimiento que necesita organizar los turnos de sus pacientes. Se parte de la premisa que el consultorio enfrenta problemas porque se cruzan los horarios y pierden el rastro del historial clínico de las personas.

2. Contexto del Problema

Elegí desarrollar este sistema porque representa un escenario cotidiano y altamente demandado, lo que lo convierte en un caso de estudio ideal para estructurar datos y relaciones de manera lógica.

Para lograr esta solución, se construye un prototipo digital que gestiona médicos, pacientes y citas, aplicando los siguientes conceptos de la programación orientada a objetos:

- **Clases:** Se crearon como plantillas para definir atributos y los métodos de cada entidad.
- **Objetos:** Se generaron instancias concretas a partir de las clases para simular médicos, pacientes y citas reales.
- **Constructores:** Se implementó el método `__init__` en cada clase para inicializar correctamente los atributos al momento de crear un objeto.
- **Métodos:** Se definieron funciones internas en las clases que dictan qué acciones puede realizar cada entidad.
- **Interacción entre objetos:** Se diseñó para que los objetos se comuniquen entre sí; por ejemplo, al crear una cita, esta reduce automáticamente la disponibilidad del médico asignado.

3. Objetivos

Objetivo general

- Desarrollar un sistema de agendamiento de citas médicas usando programación orientada a objetos.

Objetivos específicos

- Crear clases y objetos que representen fielmente a las entidades del consultorio.
- Aplicar constructores para la correcta inicialización de los datos.
- Implementar métodos que encapsulen la lógica de negocio y validaciones del sistema.
- Simular procesos reales de agendamiento y atención médica mediante un script principal.

4. Descripción del Sistema

El Sistema de agendamiento de citas médicas es una herramienta en consola que permite llevar el control integral de las atenciones en un consultorio.

El sistema inicia registrando a los médicos con su especialidad y una cantidad de turnos disponibles. Paralelamente, registra a los pacientes con sus datos básicos y un historial clínico vacío por defecto. Cuando un paciente requiere atención, el sistema genera una cita que los vincula.

Automáticamente, el sistema descuenta un turno de la agenda del médico. Luego, cuando se realiza la cita, el médico emite un diagnóstico que se guarda directamente en el historial clínico del paciente, finalizando así el ciclo de atención.

Entidades que maneja:

- **Paciente:** Usuarios que buscan atención médica.
- **Médico:** Profesionales de la salud que prestan el servicio.
- **Cita:** Entidad transaccional que relaciona a un paciente con un médico en una fecha específica.

5. Explicación de las Clases

Clase Paciente

Representa a los usuarios que asisten al consultorio en busca de atención.

- **Atributos:**
 - nombre (Texto)
 - documento (Texto)
 - historial_clinico (Lista vacía por defecto)

- **Métodos:**

- `agregar_al_historial()`: Recibe un texto y lo añade a la lista del historial clínico.
- `mostrar_historial()`: Recorre la lista de historial clínico y la imprime en pantalla. Si está vacía, muestra un mensaje indicando que no hay registros.

Clase Medico

Representa a los profesionales de la salud que atienden en el consultorio.

- **Atributos:**

- `nombre` (Texto)
- `especialidad` (Texto)
- `turnos_disponibles` (Número entero)

- **Métodos:**

- `reducir_disponibilidad()`: Resta 1 a los turnos disponibles (validando previamente que no baje de cero).
- `mostrar_info()`: Imprime en consola los datos del médico y cuántos turnos le quedan.

Clase Cita

Representa el evento de atención y actúa como un controlador entre el paciente y el médico.

- **Atributos:**

- `medico_asignado` (Objeto de la clase Medico)
- `paciente` (Objeto de la clase Paciente)
- `fecha` (Texto)
- `estado_activa` (Booleano)

- **Métodos:**

- `realizar_cita()`: Valida si la cita está activa. Si lo está, guarda el diagnóstico en el historial del paciente y desactiva la cita.
- `cancelar_cita()`: Cambia el estado de la cita a inactiva y notifica la cancelación.

5. Conclusiones

- **Importancia de la programación orientada a objetos (POO)**: el desarrollo de este modelo demostró ser una solución convincente y estructurada para abordar problemas complejos a medida que el código se expande y se vuelve más difícil de gestionar de forma puramente procedimental. Al permitirnos

interpretar y modelar el entorno real de forma exacta a cómo funciona, la POO simplificó la traducción de los requerimientos de un consultorio médico a un entorno digital transaccional.

- **Reutilización y mantenimiento del código:** Las clases funcionaron de forma efectiva como paquetes lógicos que integran atributos y métodos en una sola unidad reutilizable. Gracias a esto, el sistema permite instanciar múltiples objetos independientes sin necesidad de duplicar líneas de código, lo que eleva la calidad del software y asegura que sea flexible a modificaciones o actualizaciones futuras exigidas por el negocio.
- **Interacción e interdependencia entre clases:** se evidenció de manera práctica que el valor de la arquitectura orientada a objetos no radica en el aislamiento de sus componentes, sino en la relación y comunicación directa que se establece entre ellos. La clase Cita sirvió como un conector transaccional que interactúa estrechamente con instancias de Médico y Paciente, invocando sus respectivos comportamientos para cumplir con un flujo de trabajo colectivo y coordinado.
- **Automatización de procesos y codificación defensiva:** Al delegar responsabilidades lógicas específicas a cada objeto, se automatizaron procesos críticos que reducen la posibilidad de fallos humanos en el consultorio. Acciones como descontar automáticamente la disponibilidad de la agenda de un médico al crearse una cita, o restringir la visualización de un historial si la colección está vacía, son reflejo de la implementación de medidas de codificación defensiva.